Liver Tumor Segmentation

Submitted By

Aqib Mehmood

Session (2022-2026)

Department of Data Science

Khawaja Fareed University of Engineering & Information Technology

Rahim Yar Khan

2025

Liver Tumor Segmentation

Submitted to
Mr. Muhammad Waleed
Department of Data Science & Artificial Intelligence

In partial fulfilment of the requirements
For the degree of

Bachelor of Science in Artificial Intelligence

By

Aqib Mehmood

COSC222103013

Session (2022-2026)

Khawaja Fareed University of Engineering & Information Technology
Rahim Yar Khan
2025

DECLARATION

I hereby declare that this project report is based on my original work except for citations and quotations which have been duly acknowledged. I also declare that it has not been previously and concurrently submitted for any other degree or award at Khawaja Fareed University of Engineering & Information Technology or other institutions.

Reg No : COSC222103013

Name : Aqib Mehmood

Signature : _____

Date : _____

APPROVAL FOR SUBMISSION

I certify that this project report entitled ***Liver Tumor Segmentation*** was prepared by **Aqib Mehmood** has met the required standard for submission in partial fulfilment of the requirements for the award of Bachelors of Science in Artificial Intelligence at Khawaja Fareed University of Engineering & Information Technology.

Approved by:

Signature : _____

Supervisor: Mr. Muhammad Waleed

Date : _____

MEETING LOG

Project Name: Liver Tumor Segmentation **Supervisor:** Mr. Muhammad Waleed

Meeting No.	Date	Outcome	Signature of Supervisor
1	10/02/2025	We have discussed the project idea.	
2	12/02/2025	We confirmed the project and discussed the functional requirements in our project.	
3	18/02/2025	Discussed the documentation and further project proceedings.	
4	7/03/2025	In this meeting, chapter 1 complete documentation is presented.	
5	17/03/2025	In this meeting, chapter 2 complete documentation is presented.	
6	19/3/2025	In this meeting, Chapter 2 is reviewed.	
7	19/03/2025	We discussed the third chapter of the documentation.	
8	26/03/2025	In this meeting, chapter 3 is presented.	
9	30/03/2025	This meeting is about correcting mistakes in the chapter 3.	
10	30/03/2025	We discussed the chapter 4 of the documentation.	
11	3/04/2025	In this meeting, selecting the software process model.	
12	05/04/2025	This meeting is about diagrams of the project.	
13	11/04/2025	In this meeting, checking the UML Diagrams	
14	17/04/2025	Final draft of chapter 4 is reviewed.	
15	23/04/2025	In this meeting, final documentation is reviewed.	
16	28/04/2025	We checked our FYP-1 presentation	
17	09/09/2025	In this meeting, project scope and objectives were discussed.	

18	15/09/2025	We discussed dataset selection and preprocessing techniques for liver tumor segmentation.	
19	04/10/2025	In this meeting, the V-Net model architecture was discussed in detail.	
20	25/10/2025	Progress of model implementation and preprocessing results were reviewed.	
21	05/11/2025	Chapter 5 (Development and Implementation) complete documentation was presented.	
22	20/11/2025	Final documentation of FYP-2 was reviewed.	
23	08/12/2025	FYP-2 final presentation preparation and review were completed.	

ACKNOWLEDGEMENT

I would like to thank everyone who had contributed to this project. I would like to express my gratitude to my Project supervisor, **Mr. *Muhammad Waleed*** for his invaluable advice, guidance and his enormous patience throughout the development of the project.

In addition, I would also like to express our gratitude to my loving parents and friends who had helped and given me encouragement.

ABSTRACT

Accurate segmentation of the liver and its tumors plays a crucial role in the diagnosis, treatment planning, and post-operative assessment of hepatic diseases. However, manual delineation of tumor boundaries from volumetric CT scans is a time-consuming and error-prone process that requires significant radiological expertise. This project presents an automated liver and tumor segmentation system based on a three-dimensional V-Net architecture, designed to enhance the reliability and efficiency of clinical workflows. The proposed system processes raw abdominal CT volumes using a structured preprocessing pipeline that includes Hounsfield Unit windowing, volumetric resizing, normalization, and segmentation mask alignment. The preprocessed data is then used to train a 3D V-Net model capable of learning spatial features across depth slices, enabling robust detection of both liver regions and tumorous lesions. The model outputs a multi-class segmentation map distinguishing background, liver tissue, and tumor regions. To support practical applicability, an interactive desktop application has been developed using Tkinter. The application allows clinicians or researchers to upload CT scans, run the segmentation model, and visualize predicted liver and tumor boundaries overlaid on the original slices. This end-to-end system demonstrates the feasibility of deploying volumetric deep learning techniques for medical image segmentation in real-world scenarios.

TABLE OF CONTENTS

CHAPTER 1 ..... **1**

INTRODUCTION **1**

1.1. Background 1

1.2. Significance in Medical Imaging 1

1.2.1. Importance of Liver Tumor Detection 2

1.2.1.1. Role of Deep Learning in Tumor Segmentation *2*

1.3. Introduction to Project Goals 2

1.4. Problem Statement 3

1.5. Objective 3

1.6. Project Scope 4

1.7. Advantages of the System 5

1.8. Relevance to Data Science and Web Development 6

1.9. Chapter Summary 6

CHAPTER 2 ..... **7**

EXISTING SYSTEMS **7**

2.1. Related Studies in Medical Image Segmentation 7

2.2. Drawbacks in Related Works 7

2.3. Research Gaps in Tumor Segmentation 8

2.4. Comparison of Related Works 8

2.5. Chapter Summary 8

CHAPTER 3 ..... **10**

REQUIREMENT ENGINEERING **10**

3.1. Proposed System 10

3.2. Understanding the System 10
3.2.1. User Roles and Involvement 10
3.2.2. Stakeholders 10
3.2.3. Domain: Medical Image Analysis 11
3.2.4. System Needs 11
3.3. Requirement Engineering 11
3.3.1. Functional Requirements 12
3.3.2. Non-Functional Requirements 14
3.3.3. Requirements Baseline 16
3.3.4. Need to Feature Mapping 17
3.4. Gantt chart 18
3.5. Hurdles in Optimizing the Current System 18
3.6. Chapter Summary 19
CHAPTER 4 20
DESIGN 20
4.1. Selected Software Process Model 20
Figure 4.1 Iterative Model Diagram 20
4.1.1. Benefits of Selected Model 20
4.1.2. Limitations of Selected Model 21
4.2. Design 21
4.2.1. V-Net or Deep Learning Algorithm Key Characteristics of V-net: 21
4.2.2. Methodology: Preprocessing, Training, Prediction 23
4.2.3. Entity Relationship Diagram 24
4.2.4. UML Diagrams 25
4.2.4.1. Use Case Diagram of the System 25

4.2.4.2. Class Diagram of the System *26*

4.2.4.3. Activity Diagram of the System *27*

4.2.4.4. Sequence Diagram of the System *28*

4.2.4.5. Component Diagram of the System *29*

4.3. Chapter Summary 29

CHAPTER 5 ..... **30**

DEVELOPMENT AND IMPLEMENTATION **30**

5.1. Development of the Computer Program 30

5.2. Implementation Strategy 31

5.3. Tools Selection 32

5.3.1. Hardware 32

5.3.2. Software 32

5.4. Coding 33

5.5. Description 33

5.6. Program Deployment 35

5.7. Chapter Summary 36

CHAPTER 6 ..... **37**

TESTING **37**

6.1. Introduction 37

6.2. Testing Methods 37

6.3. Comparison 38

6.4. Software Evaluation 39

6.4.1. Testing Strategy 39

6.4.2. Test Plans 39

6.4.3. Test Cases 40

6.4.4. Test Report 41

6.5. Chapter Summary 41

REFERENCES 42

LIST OF FIGURES

FIGURE 3.1 GANTT CHART 18

FIGURE 4.1 ITERATIVE MODEL DIAGRAM 20

FIGURE 4.2 METHODOLOGY DIAGRAM 23

FIGURE 4.3 ENTITY RELATIONSHIP (ER) DIAGRAM 24

FIGURE 4.4 USE CASE DIAGRAM 25

FIGURE 4.5 CLASS DIAGRAM 26

FIGURE 4.6 ACTIVITY DIAGRAM 27

FIGURE 4.7 SEQUENCE DIAGRAM 28

FIGURE 4.8 COMPONENT DIAGRAM 29

FIGURE 5.1 INTERFACE DIAGRAM 35

LIST OF TABLES

TABLE 3.1 SYSTEM NEEDS 11

TABLE 3.2 FUNCTIONAL REQUIREMENT 01 12

TABLE 3.3 FUNCTIONAL REQUIREMENT 02 13

TABLE 3.4 FUNCTIONAL REQUIREMENT 03 13

TABLE 3.5 FUNCTIONAL REQUIREMENT 04 14

TABLE 3.6 FUNCTIONAL REQUIREMENT 05 14

TABLE 3.3.4 FEATURE MAPPING 14

TABLE 6.1 COMPERISON TABLE 40

Chapter 1

INTRODUCTION

1.1. Background

Liver cancer is one of the most prevalent and life-threatening diseases worldwide, particularly in regions with high rates of hepatitis infections and chronic liver diseases. Among all types of liver cancers, hepatocellular carcinoma (HCC) is the most common. Early diagnosis and accurate segmentation of liver tumors play a critical role in improving treatment outcomes and patient survival rates. However, manual analysis and segmentation of CT or MRI liver scans by radiologists are time consuming, subjective, and prone to human error. With the increasing availability of large-scale annotated medical imaging datasets and advancements in artificial intelligence, particularly deep learning, automated liver tumor segmentation has gained significant attention in the medical research and healthcare communities. Deep learning models, especially Convolutional Neural Networks (CNNs), have shown great success in image recognition and medical image analysis tasks due to their ability to learn complex patterns and features from large datasets.

1.2. Significance in Medical Imaging

Medical imaging plays a central role in modern diagnostic and therapeutic workflows. It serves as a non-invasive window into the internal structures and functions of the human body, enabling physicians to detect, characterize, and monitor diseases in ways that were previously impossible. Among the various organs examined through imaging modalities, the liver poses unique challenges due to its complex vascular structure, heterogeneous tissue composition, and susceptibility to a wide range of pathologies, including cancer. Detecting liver tumors early and accurately is vital because it significantly increases the likelihood of successful treatment, prolongs survival rates, and improves the overall quality of patient care.

The segmentation of liver tumors from medical images is a task of immense clinical importance. Segmentation refers to the process of delineating the exact boundaries of the tumor within the liver from medical scans. This process is fundamental for multiple reasons: it helps in calculating the tumor volume, assessing its growth or shrinkage over time, planning surgeries or radiation therapy, and evaluating the response to treatment. Without precise segmentation, there is a higher risk of incomplete tumor removal during surgery,

under- or over-estimation of tumor burden, and inaccurate targeting during radiation therapy, all of which could have serious clinical implications.

1.2.1. Importance of Liver Tumor Detection

Detecting liver tumors accurately and at an early stage is critical in improving patient survival and treatment outcomes. The liver is one of the most vital organs in the human body, responsible for essential functions such as detoxification, metabolism, and the synthesis of important proteins. Due to its high blood flow and complex anatomy, the liver is also a common site for both primary tumors (such as hepatocellular carcinoma) and secondary metastases originating from other organs. Early identification of these tumors enables timely medical intervention, which can significantly reduce the risk of disease progression and complications.

Liver tumors are often asymptomatic in their early stages, making early detection through imaging essential. When tumors are discovered at a late stage, treatment options become limited and less effective, often resulting in poor prognosis. Imaging technologies such as CT and MRI scans allow clinicians to visualize liver tissues in high detail, making it possible to detect abnormalities even before symptoms arise. However, the effectiveness of these methods depends heavily on the precision and reliability of tumor detection and delineation.

1.2.1.1. *Role of Deep Learning in Tumor Segmentation*

Deep learning has revolutionized the field of medical imaging by providing highly accurate and automated solutions for complex tasks such as liver tumor segmentation. Unlike traditional image processing techniques that rely on hand crafted features and manual intervention, deep learning models learn hierarchical patterns directly from large datasets, making them well-suited for identifying tumors in complex imaging modalities like CT and MRI scans.

1.3. Introduction to Project Goals

The primary goal of this project is to develop an intelligent and efficient liver tumor segmentation system using deep learning techniques. This system is designed to analyse medical imaging data specifically CT scans to detect and segment liver tumors accurately and automatically. The objective is not only to assist radiologists in identifying tumors with

greater precision but also to streamline the diagnostic workflow, reduce human error, and improve the consistency of medical image interpretations.

By leveraging advanced deep learning algorithms such as V-net, the project aims to create a robust segmentation model that can learn from annotated datasets and generalize to new patient scans. The ultimate output is a tool that can highlight tumor regions in liver scans, helping clinicians make quicker and more informed decisions.

Overall, this project seeks to contribute meaningfully to the medical imaging field by combining the power of artificial intelligence with accessible software design, thereby supporting early tumor detection and improving outcomes for liver cancer patients.

1.4. Problem Statement

Liver cancer remains one of the leading causes of cancer-related deaths worldwide, largely due to the difficulty in detecting tumors at an early stage. Accurate and timely segmentation of liver tumors from medical images such as CT scans is critical for effective diagnosis, staging, and treatment planning. However, manual segmentation by radiologists is a laborious process that is time consuming and susceptible to human error and inter observer variability.

The complexity of liver anatomy, presence of artifacts in medical images, and variations in tumor shape, size, and intensity make the segmentation task even more challenging. In resource constrained environments where access to highly skilled radiologists is limited, these challenges can lead to delayed or inaccurate diagnoses, which may compromise patient outcomes.

Existing automated segmentation tools often suffer from limited generalization, lack of robustness, and difficulty in handling noisy and low contrast images. Additionally, many of these systems are not adaptable to diverse datasets and real-time clinical environments. There is a clear need for an intelligent, efficient, and reliable solution that can perform liver tumor segmentation with high precision and minimal human intervention.

1.5. Objective

The main objective of this project is to develop an intelligent and automated system for liver tumor segmentation using deep learning techniques, specifically targeting CT scan

images. The system aims to support radiologists and healthcare professionals by providing accurate, consistent, and fast segmentation of liver tumors, reducing the reliance on manual efforts and enhancing clinical decision making.

The specific objectives of the project include:

- To design and implement a deep learning based segmentation model using architectures such as V-net or its variants for identifying liver tumors in CT images.
- To preprocess and enhance medical imaging data for improved model training and performance, including noise reduction, normalization, and data augmentation.
- To train the segmentation model on publicly available medical datasets like the LiTS (Liver Tumor Segmentation Challenge) dataset for accurate tumor boundary detection.
- To evaluate the system's performance using standard metrics such as accuracy, Dice coefficient, precision, recall, and F1-score.
- To reduce diagnostic time and variability in tumor detection through consistent, high precision automated segmentation.
- To design a user-friendly interface or output format that can assist medical professionals in visualizing and interpreting the segmentation results effectively.
- To contribute to the advancement of AI-based clinical tools in the field of radiology and medical imaging.

1.6. Project Scope

The scope of this project focuses on the development of an automated liver tumor segmentation system using deep learning techniques, specifically targeting CT scan image analysis. The system is designed to assist radiologists by identifying and delineating liver tumors from CT images with minimal manual intervention. The model will be trained using a publicly available dataset and tested to evaluate its performance based on real world medical imaging conditions.

The project encompasses various stages, including data preprocessing, model training and validation, performance evaluation, and visualization of results. It covers tumor segmentation only and does not extend to classification of tumor types or post-segmentation treatment recommendations. This system will be beneficial in hospital settings, research environments,

and diagnostic centers, especially where there is a shortage of experienced radiologists or a high volume of imaging data. It is expected to significantly improve the speed and consistency of tumor analysis, making it a valuable tool in clinical workflows and medical research.

1.7. Advantages of the System

The proposed liver tumor segmentation system provides several advantages over traditional manual diagnostic approaches and existing tools. By utilizing deep learning based automation, the system enhances the accuracy, speed, and reliability of liver tumor detection and segmentation from CT scans.

- **Improved Accuracy**: The system uses deep learning models trained on medical datasets to achieve high precision in segmenting liver tumors, reducing the likelihood of missing or misidentifying tumor regions.
- **Time Efficiency**: Automated segmentation significantly reduces the time required for image analysis compared to manual methods, enabling faster diagnosis and treatment planning.
- **Reduced Human Error**: The system eliminates inconsistencies caused by fatigue or human interpretation variability, ensuring standardized and repeatable results.
- **Support for Radiologists**: By highlighting tumor areas, the system acts as a supportive tool for radiologists, allowing them to validate the AI-generated results and focus on complex decision-making tasks.
- **Scalability**: The system is designed to handle large volumes of imaging data, making it suitable for use in hospitals, research institutions, and diagnostic labs.
- **Cost-Effective**: Once deployed, the system reduces the need for intensive manual labor, ultimately lowering the overall operational costs in healthcare environments.
- **Data Retention and Analysis**: The system stores segmented results which can be used for follow-up comparisons, long-term monitoring, and clinical research purposes.
- **Educational Use**: The system can be used as a training tool for medical students and junior radiologists to learn about tumor patterns and segmentation techniques.

1.8. Relevance to Data Science and Web Development

The Liver Tumor Segmentation project is closely aligned with the core concepts and technologies taught throughout the Bachelor of Science in Computer Data Science/Artificial Intelligence program. It is a multidisciplinary project that incorporates knowledge from various subjects including Artificial Intelligence, Deep Learning, Image Processing, Programming Fundamentals, Software Engineering, and Data Science.

The system applies machine learning and deep learning principles to solve real-world medical problems, reflecting the practical applications of theoretical concepts studied in the coursework. It encourages the use of project based learning and promotes critical thinking in the areas of healthcare automation and biomedical computing.

From software development to model training, the project emphasizes problem solving skills, algorithm design, and system optimization key learning outcomes expected from the degree program.

1.9. Chapter Summary

This chapter provided an overview of the project titled "Liver Tumor Segmentation" and highlighted its relevance in the field of medical imaging and healthcare automation. It began with the background of liver cancer diagnosis and emphasized the importance of accurate tumor segmentation using advanced AI technologies. The chapter also discussed the significance of deep learning in medical imaging, particularly its role in automating radiological tasks and reducing human dependency.

Furthermore, the chapter presented a detailed problem statement, defined the objectives of the system, and outlined the overall scope and potential impact of the project. It highlighted the advantages of the proposed system in terms of performance, reliability, and real world applicability. Lastly, the connection between this project and the academic curriculum was established, demonstrating how this work builds upon knowledge and skills acquired throughout the degree program.

Chapter 2

EXISTING SYSTEMS

2.1. Related Studies in Medical Image Segmentation

Medical image segmentation has evolved significantly over the last two decades, driven by the need for precision in diagnosis and treatment planning. Segmentation involves partitioning an image into meaningful regions, typically separating organs, tissues, or pathological areas such as tumors. Traditional segmentation techniques were largely reliant on manual annotation or classical image processing methods like thresholding, region growing, or edge detection. While these methods offered initial value, they often lacked robustness across diverse patient data and imaging conditions.

The breakthrough in segmentation came with the advent of machine learning and, more recently, deep learning. Machine learning based segmentation models, such as Support Vector Machines (SVMs) and Random Forests, allowed automated feature extraction but still required careful engineering of those features. However, deep learning revolutionized the field by automating feature learning, especially with the use of Convolutional Neural Networks (CNNs).

2.2. Drawbacks in Related Works

Despite the rapid advancement in medical image segmentation, existing research and methodologies still face several notable challenges and limitations, especially when applied to liver tumor segmentation. These drawbacks hinder their direct clinical adoption and limit their performance in real-world applications.

One of the most pressing issues is limited generalization. Many deep learning models, though highly accurate on benchmark datasets, perform poorly when tested on unseen data from different sources or hospitals. This is due to the variability in image acquisition protocols, scanner types, and patient anatomy. For instance, liver tumors can vary significantly in size, shape, and intensity, which causes models trained on one dataset (like LiTS) to struggle with images from another source.

Another significant drawback is the dependency on large annotated datasets. Deep learning models require thousands of well-labelled images to learn effectively. However,

medical image annotation is time-consuming and demands expert radiologists, making such datasets rare and expensive. Furthermore, in liver tumor segmentation, accurate annotation is particularly challenging due to the ambiguous boundaries of some tumors, especially in low contrast regions of CT scans.

2.3. Research Gaps in Tumor Segmentation

While a vast body of research exists in the field of medical image segmentation, particularly using deep learning, several key research gaps continue to limit the practical effectiveness and clinical adoption of liver tumor segmentation systems.

A major gap lies in the insufficient diversity of training datasets. Many segmentation models are trained and validated on publicly available datasets like LiTS or MICCAI, which, while valuable, may not capture the full heterogeneity of real world liver tumor cases. These datasets often include limited demographic diversity, disease stages, and imaging artifacts. Consequently, models may underperform when deployed in clinical environments where data characteristics differ significantly.

2.4. Comparison of Related Works

Comparing existing studies in medical image segmentation particularly those focused on liver tumor detection reveals diverse approaches, models, and outcomes, each with unique strengths and limitations. This comparison highlights how the field has evolved and where our project aims to improve.

One of the foundational models in biomedical segmentation is V-net, introduced in 2015. It set the benchmark for many subsequent models due to its symmetric encoder-decoder structure and skip connections, which help retain spatial context. Studies using the original V-net have reported high Dice coefficients (a common segmentation metric), especially on structured datasets like LiTS. However, V-net struggles when handling highly irregular tumor shapes or varying imaging conditions.

2.5. Chapter Summary

This chapter provided an overview of the existing systems in the field of liver tumor segmentation, highlighting their capabilities and limitations. Various current techniques were examined, with a focus on their reliance on traditional image processing methods, which often require extensive manual intervention and expert knowledge. While these systems show

promise, they tend to struggle with issues such as inconsistent results, low accuracy, and the inability to handle complex tumor shapes.

Additionally, this chapter identified the drawbacks within these existing systems, including their lack of automation, which can lead to delays and increased human error. It was concluded that there is a clear need for more advanced systems, particularly those leveraging deep learning and artificial intelligence, which can automate the segmentation process and improve both speed and accuracy.

Chapter 3

REQUIREMENT ENGINEERING

3.1. Proposed System

The proposed system focuses on developing an automated liver tumor segmentation tool using deep learning techniques. The primary objective of the system is to assist radiologists and medical professionals in identifying and segmenting liver tumors from CT scan images with higher accuracy and efficiency compared to traditional methods.

The system will utilize a convolutional neural network (CNN) architecture specifically, a V-net-based model which has proven effective in biomedical image segmentation tasks. The model will be trained on publicly available datasets such as the Liver Tumor Segmentation Challenge (LiTS) dataset. The dataset includes annotated CT scans, which will be used to teach the model to differentiate between liver tissue and tumor regions.

3.2. Understanding the System

Understanding the system is essential for aligning development objectives with user needs and technical feasibility. This section elaborates on how the system interacts with various stake holders, its operational domain, and the specific requirements it aims to fulfil.

3.2.1. User Roles and Involvement

The primary users of this system include radiologists, healthcare professionals, and researchers. Their role is to upload CT images and interpret the segmentation results for diagnostic or analytical purposes. The system minimizes user involvement in the technical process, allowing non-technical users to benefit from AI-powered analysis.

3.2.2. Stakeholders

Key stakeholders include medical institutions, radiology departments, AI researchers in healthcare, and software developers involved in the system's maintenance. Stakeholders benefit through improved diagnostic tools, research support, and technological innovation in medical imaging.

3.2.3. Domain: Medical Image Analysis

This project lies in the domain of medical image analysis, specifically focusing on semantic segmentation within radiology. The segmentation of liver tumors helps in early detection, treatment planning, and progress tracking, making it highly relevant in oncological practices.

3.2.4. System Needs

Table 3.1 System Needs

SR #	Needs	Need ID
1	GPU-enabled system for model training	N-01
2	Sufficient RAM (≥16GB) and storage (≥100GB)	N-02
3	Python with Tensor Flow/PyTorch and OpenCV	N-03
4	Django framework with HTML, CSS, and JavaScript	N-04
5	Annotated medical datasets (e.g., LiTS)	N-05
6	SQLite database for backend data storage	N-06
7	Visualization tools like Matplotlib or PIL	N-07
8	Internet for deployment or cloud hosting	N-08

3.3. Requirement Engineering

Requirement engineering is the foundational process of identifying, analysing, documenting, and validating the needs of a software system. It ensures that the developed system aligns with user expectations and project goals. For the liver tumor segmentation system, requirements were collected by understanding medical imaging challenges, user roles (e.g., doctors, radiologists, administrators), and the technical feasibility of deploying AI-driven tools.

In this project, both functional and non-functional requirements have been carefully identified. Functional requirements define what the system should do such as uploading CT images, running segmentation, and displaying results. Non-functional requirements focus on how the system performs like responsiveness, usability, and security in handling sensitive medical data.

3.3.1. Functional Requirements

In this project, both functional and non-functional requirements have been carefully identified. Functional requirements define what the system should do such as uploading CT images, running segmentation, and displaying results. Non-functional requirements focus on how the system performs like responsiveness, usability, and security in handling sensitive medical data.

Table 3.2 Functional Requirement 01

Functional Requirement ID	FR-01
Name	Access Educational Pages
Description	Users should be able to view informational content on liver tumors such as causes, symptoms, diagnosis, and treatment.
Input	Page navigation click
Output	Educational content rendered
Precondition	User accesses the site via browser
Postcondition	Page loads with relevant medical information displayed

Table 3.3 Functional Requirement 02

Functional Requirement ID	FR-02
Name	Upload CT Scan Image
Description	Users can upload liver CT scan images for segmentation.
Input	CT image file (.png, .jpg, etc.)
Output	Image uploaded to the server
Precondition	User is on the prediction/upload section
Postcondition	Image is stored and queued for model processing

Table 3.4 Functional Requirement 03

Functional Requirement ID	FR-03
Name	Run Segmentation Model
Description	The backend processes the uploaded image using the trained model to segment the liver tumor region.
Input	Uploaded image
Output	Segmented image with tumor highlighted
Precondition	Image has been uploaded
Postcondition	Prediction result is generated and saved

Table 3.5 Functional Requirement 04

Functional Requirement ID	FR-04
Name	Display Prediction Results
Description	The segmented image is displayed on the frontend for user interpretation.
Input	Segmented image file
Output	Result visualization on browser
Precondition	Prediction completed
Postcondition	User views the model's output side-by-side with the input

Table 3.6 Functional Requirement 05

Functional Requirement ID	FR-05
Name	Save Session and Image Metadata
Description	Information such as user session, file path, and output is stored in the database for tracking.
Input	Session info, uploaded image, segmentation result
Output	Stored record in SQLite database
Precondition	User uploads image and segmentation completes
Postcondition	Data is stored for future access or analysis

3.3.2. Non-Functional Requirements

Non-functional requirements define the quality attributes, performance standards, and usability expectations of the liver tumor segmentation system. While functional requirements explain what the system does, non-functional requirements describe how

well it performs those tasks. Below are the key non-functional requirements for this project:

1. **Performance** :
   - The model should generate segmentation results approximately within 10-15 seconds of image upload to ensure usability in practical scenarios.
   - The web interface must load educational pages and prediction results without noticeable lag.
2. **Usability** :
   - The frontend interface is designed with simplicity to cater to medical practitioners, students, and non-technical users.
   - Buttons, navigation, and content should be intuitive and self-explanatory.
3. **Reliability**:
   - The system should handle multiple requests without crashing or producing incorrect results.
   - Any model failure (e.g., invalid image format) should be managed gracefully with proper error messages.
4. **Security**:
   - Uploaded medical images must be handled securely with temporary storage.
   - Access to prediction and upload features may require login or authentication in future versions.
5. **Scalability**:
   - The Django backend architecture allows scaling the application to handle more users or integrate more advanced models later.
6. **Maintainability**:
   - The project codebase is modular, separating frontend, backend, and model components for easy updates and debugging.
7. **Compatibility**:
   - The web application should work across common modern browsers (Chrome, Firefox, and Edge) and be mobile-responsive.

3.3.3. Requirements Baseline

The Requirements Baseline serves as a finalized agreement on the system's expected features and constraints before the actual development begins. It includes both functional and non-functional requirements that have been thoroughly discussed, documented, and reviewed. These requirements act as a foundation for the design, development, and testing phases. For the Liver Tumor Segmentation project, the following points represent the agreed-upon baseline:

- The system must allow users to upload liver CT scan images and receive segmented output indicating tumor regions.
- The system must provide a user-friendly web interface containing both informational pages and image prediction functionality.
- The segmentation model should provide accurate predictions, and the output must be clearly visualized for medical analysis.
- The system must be developed using Python, Django, HTML, CSS, and deep learning frameworks (Keras/TensorFlow or PyTorch).
- The system must adhere to non-functional quality constraints, including performance (fast response time), security (safe image handling), and usability (easy navigation and clear results).
- All requirements listed in Sections 3.3.1 (Functional) and 3.3.2 (Non-Functional) form the official scope and cannot be changed without formal revision.

This baseline ensures that all stakeholders, including the development team and potential users, have a shared understanding of what the final system is expected to deliver.

3.3.4. Need to Feature Mapping

Table 3.3.4 Feature Mapping 01

Need ID	Needs Description	Mapped Feature
N-01	Understand Liver Tumor and its Medical Background	Informative pages: About, Causes, Symptoms, Diagnosis, Treatment
N-02	Enable Image Upload for Tumor Detection	CT scan image upload feature
N-03	Deliver Accurate Tumor Segmentation	Trained deep learning model for image processing
N-04	Provide Visual Feedback to Users	Display of segmented image on the web interface
N-05	Support Decision-Making with Clean UI	Simple and clean frontend design with easy navigation
N-06	Secure Handling of Medical Data	Temporary storage and validation of uploaded images
N-07	Quick Result Generation	Fast processing and response from backend model
N-08	Long-Term Maintenance and Scalability	Modular architecture using Django and Python

3.4. Gantt chart

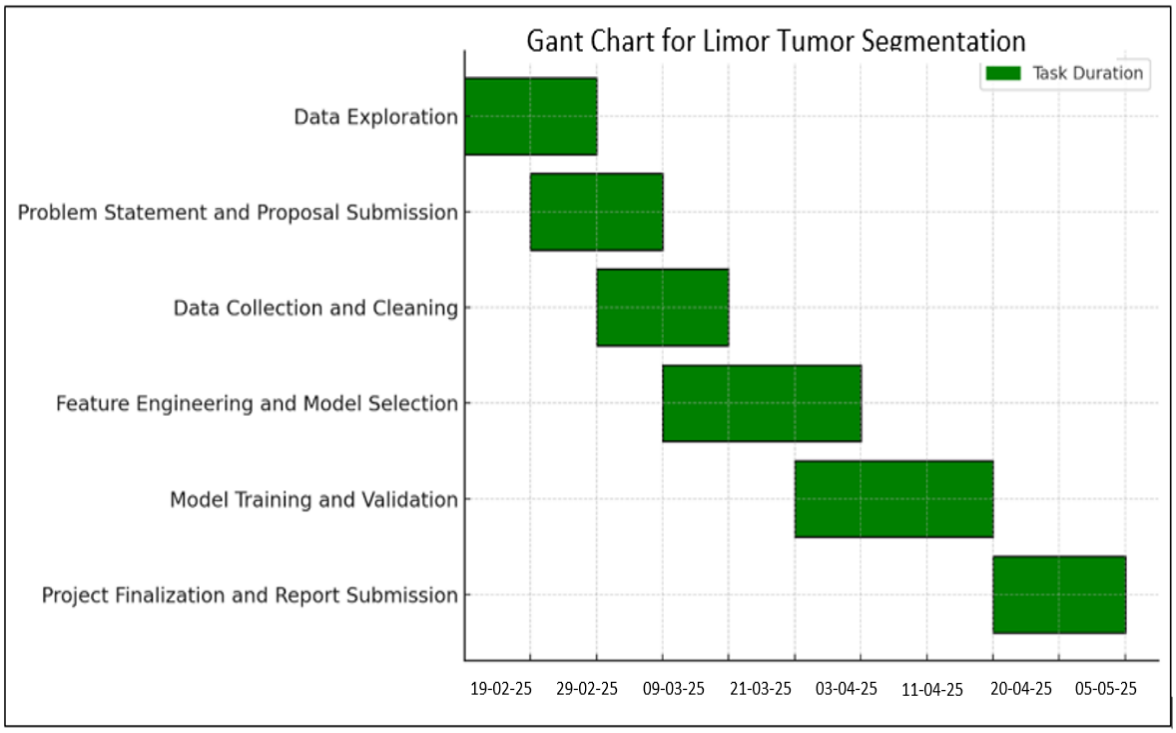

Figure 3.1 Gantt chart

3.5. Hurdles in Optimizing the Current System

While developing and deploying the Liver Tumor Segmentation system, several challenges were encountered that affected optimization and system performance. Addressing these hurdles is crucial for improving accuracy, usability, and scalability.

- **Data Quality and Annotation Issues:**

  Medical imaging datasets like LiTS often come with inconsistent or limited annotations. Some CT scans lack precise labeling, especially in edge cases with small or diffused tumors, which affects the model's learning and generalization capabilities.

- **Model Training Limitations:**

  Training deep learning models for medical segmentation requires high computational power and fine-tuning of hyper parameters. GPU memory limitations, long training times, and overfitting on a small dataset were significant hurdles that required batch size adjustments and model simplification.

- **Complexity in Tumor Detection:**

  Liver tumors vary in size, shape, and intensity across different CT images. This variability makes it hard for segmentation models to maintain consistent accuracy, especially in detecting smaller or less distinguishable tumors.

- **Web Integration Challenges:**

  Integrating the trained model into a Django-based web application posed compatibility issues. Model loading time, file size handling, and image processing on the server affected the real-time performance and required backend optimizations.

- **User Interface Constraints:**

  While the frontend interface was designed for simplicity, ensuring responsiveness across devices and maintaining clean visualization of medical results was challenging.

- **Security and Privacy Concerns:**

  Handling medical images involves sensitive data. Ensuring secure upload, temporary storage, and immediate deletion post-processing had to be addressed using proper validation and session control techniques in Django.

3.6. Chapter Summary

This chapter provided a comprehensive overview of the Liver Tumor Segmentation system from a requirement engineering perspective. It began with a detailed understanding of the proposed system, covering the user roles, stakeholders, and the domain of medical image analysis. The system needs were identified and organized to align with the core objectives of the project.

In summary, this chapter laid the foundation for a systematic and goal-oriented development process, ensuring the project moves forward with well-defined expectations and a structured plan.

Chapter 4

DESIGN

4.1. Selected Software Process Model

For the Liver Tumor Segmentation project, the Iterative Waterfall Model is the most appropriate software development methodology. Since this is an individual project with clear requirements, a structured yet flexible approach is necessary. The Iterative Waterfall Model offers a balance between traditional documentation-heavy phases and the ability to revisit and refine earlier stages based on feedback and performance testing.

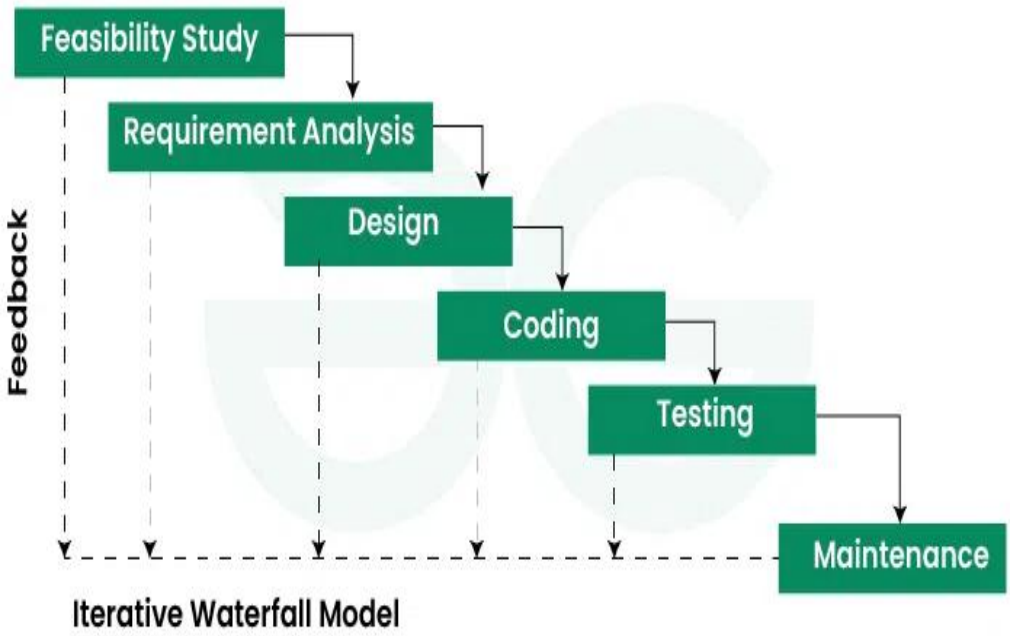

Figure 4.1 Iterative Model Diagram

4.1.1. Benefits of Selected Model

1. **Clear Stage-wise Progression:**

   Each phase such as requirement analysis, design, implementation, testing, and deployment is well-defined, making the project easier to manage individually.

2. **Early Problem Identification:**

Errors or mismatches in earlier phases (like data pre-processing or UI design) can be quickly identified and corrected in iterative cycles.

3. **Documentation Friendly:**

   This model encourages detailed documentation at each stage, aligning well with academic and project reporting requirements.

4. **Improved Quality through Feedback:**

   Iterations allow for gradual improvements, especially useful when optimizing model performance or refining frontend usability.

4.1.2. Limitations of Selected Model

1. **Time Consuming:**

   Iterating through multiple cycles may increase development time.

2. **Heavy on Planning:**

   Requires detailed upfront planning and ongoing revisions, which can be challenging without team collaboration.

3. **Rigid between Phases:**

   Though iterative, transitions between major phases still follow a sequential structure, limiting agility in drastic design changes.

4. **Dependent on Dataset Availability:**

   The model assumes the dataset and system requirements are well understood early on, which may not always be the case in deep learning projects.

4.2. Design

4.2.1. V-Net or Deep Learning Algorithm Key Characteristics of V-net:

For this project, the V-net architecture has been selected for liver tumor segmentation. V-net is a specialized convolutional neural network (CNN) primarily used for biomedical image segmentation tasks. It is well-suited for this project due to its ability to accurately localize and segment tumor regions in medical images with limited training data.

Key Characteristics of V-net:

- **Symmetric Architecture**: V-net follows an encoder-decoder structure. The left side (encoder) captures the context of the image through convolution and pooling, while the right side (decoder) performs up sampling to recover spatial resolution.
- **Skip Connections**: One of the defining features of V-net is the skip connections between encoder and decoder layers, allowing the model to retain fine-grained information necessary for precise segmentation.
- **Efficient Training on Small Datasets**: Since annotated medical images are often limited, V-net's data efficiency is highly beneficial. It performs well with data augmentation and patch-based training strategies.

Application to Liver Tumor Segmentation:

 Input: Pre-processed CT or MRI scan slices.

 Output: A binary or multi-class mask highlighting tumor regions.

 Loss Function: Dice coefficient or binary cross-entropy, optimizing for overlap between predicted and true mask.

 Performance: High accuracy in segmenting irregular and small tumor shapes due to the localization capacity of the V-net architecture.

4.2.2. Methodology: Preprocessing, Training, Prediction

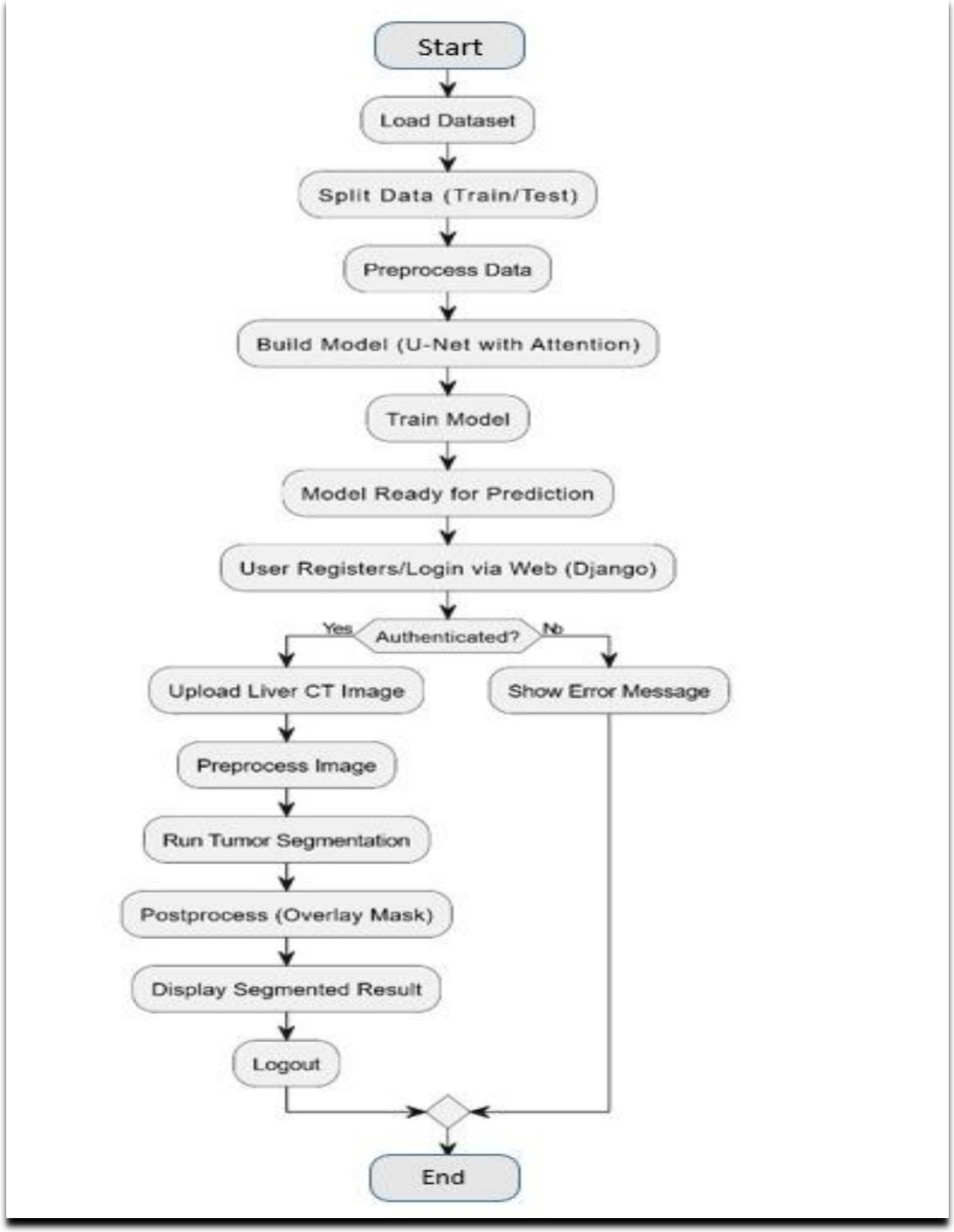

Figure 4.2 Methodology Diagram

4.2.3. Entity Relationship Diagram

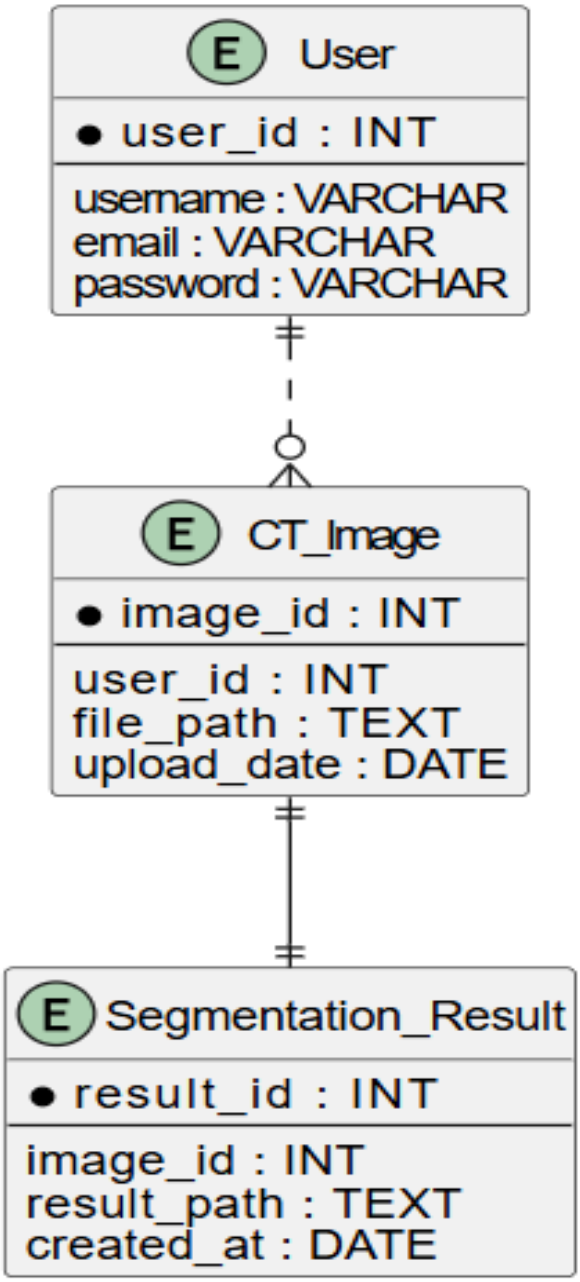

Figure 4.3 ER Diagram

4.2.4. UML Diagrams

4.2.4.1. *Use Case Diagram of the System*

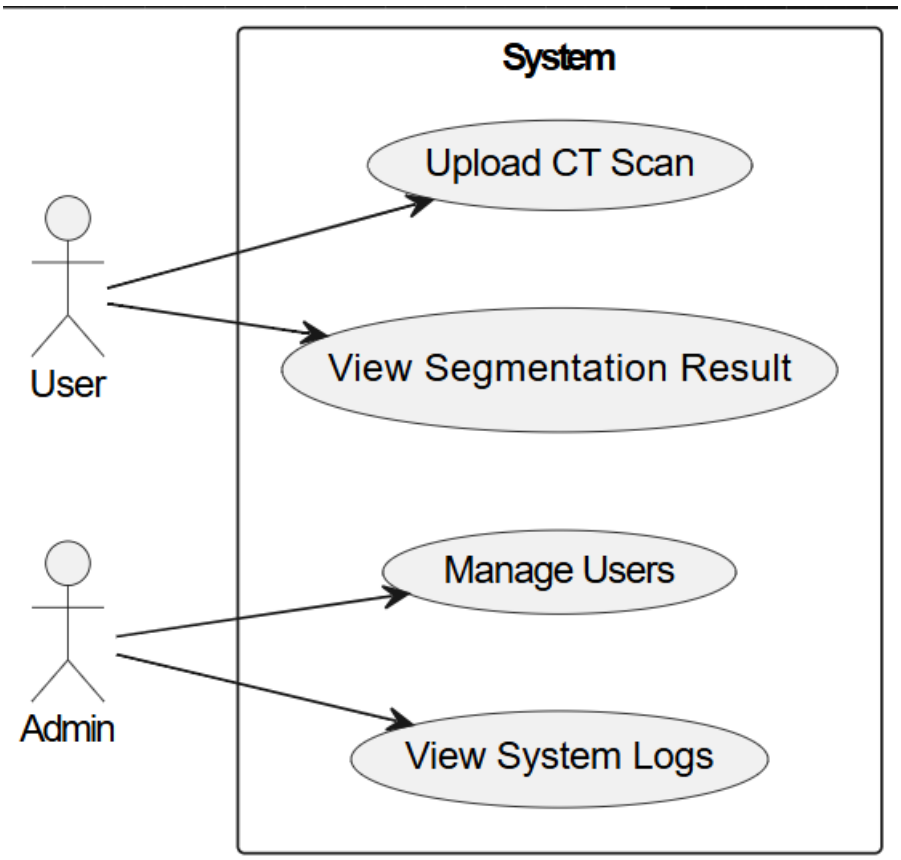

Figure 4.4 Use Case Diagram

4.2.4.2. *Class Diagram of the System*

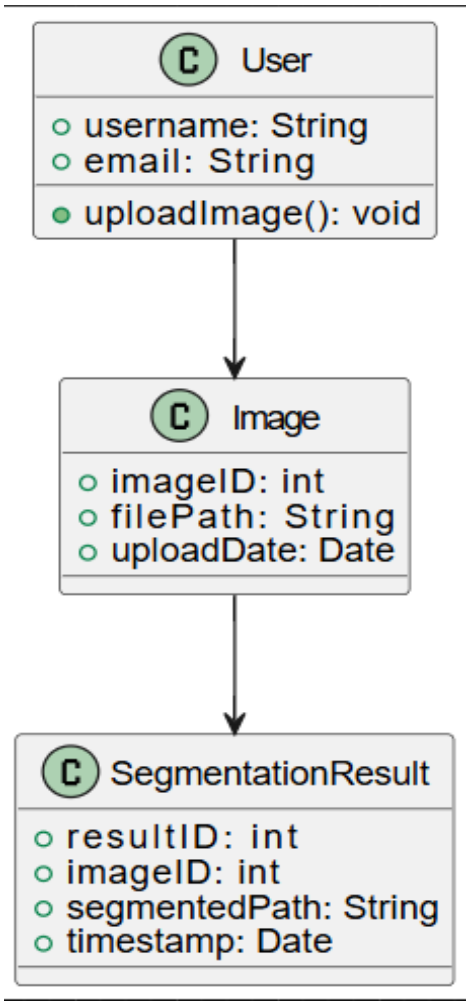

Figure 4.5 Class Diagram

4.2.4.3. *Activity Diagram of the System*

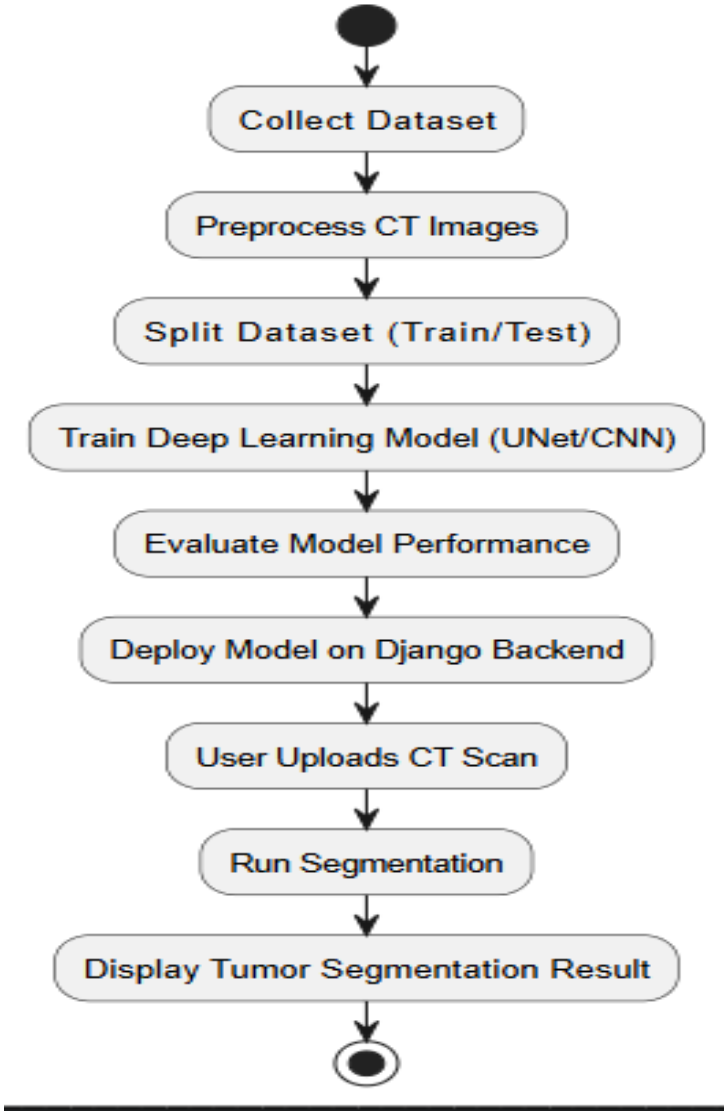

Figure 4.6 Activity Diagram

4.2.4.4. *Sequence Diagram of the System*

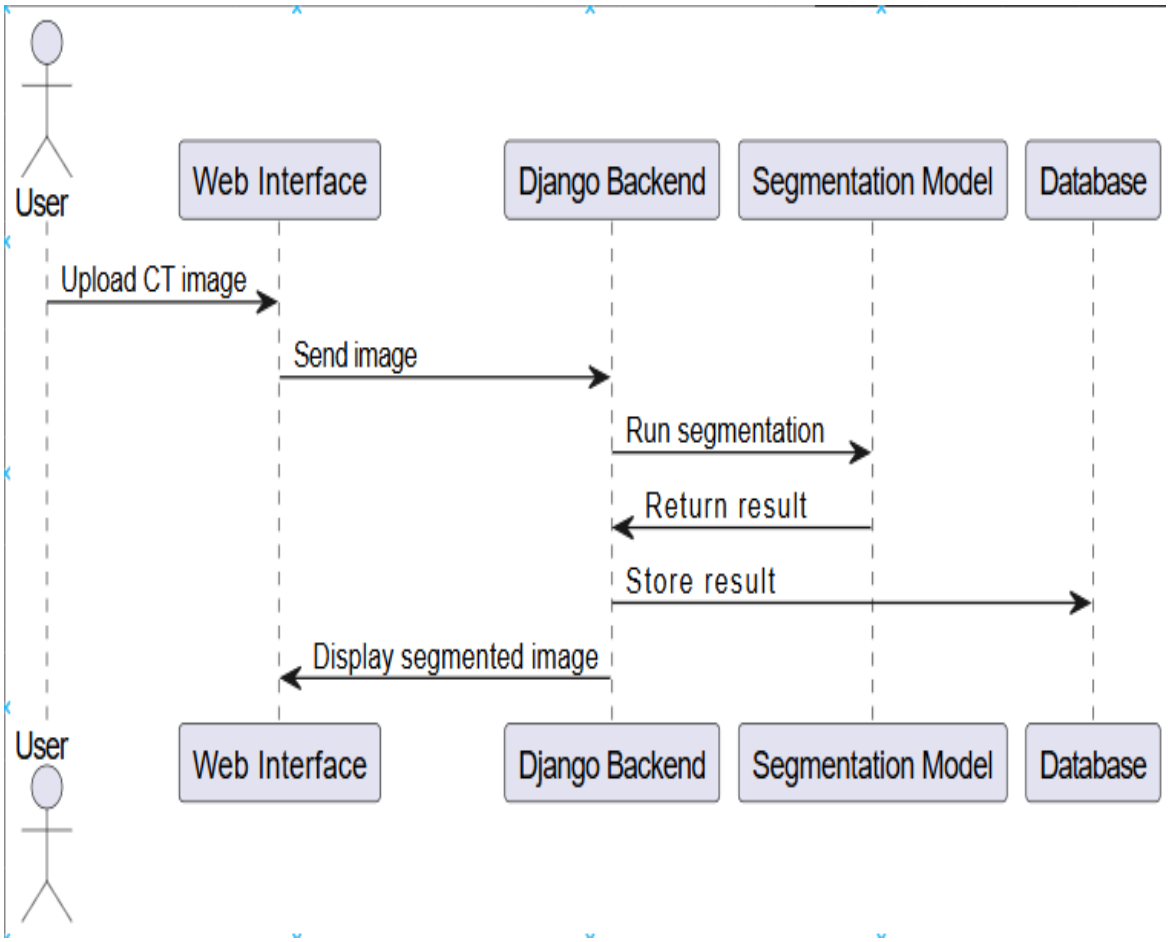

Figure 4.7 Sequence Diagram

4.2.4.5. *Component Diagram of the System*

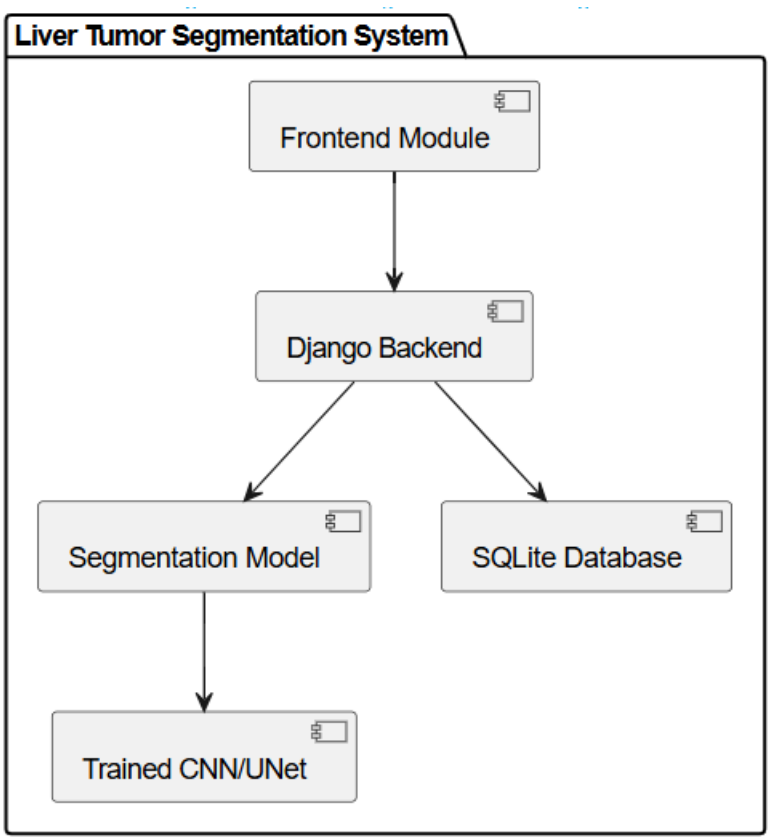

Figure 4.8 Component Diagram

4.3. Chapter Summary

In this chapter, we presented the design aspects of the Liver Tumor Segmentation system. The design phase plays a critical role in defining the structural and functional framework of the entire project.

We began by describing the System Architecture, which outlines the interaction between the user interface, backend server, deep learning model, and image processing components. This architecture ensures a smooth flow from image upload to segmentation output.

Following this, we elaborated on the Data Flow and UML diagrams that depict how data is processed and how various modules interact within the system. These diagrams serve as a blueprint for implementation and help identify potential integration challenges in advance.

Chapter 5

DEVELOPMENT AND IMPLEMENTATION

5.1. Development of the Computer Program

The development of the liver tumor segmentation system was carried out in several iterative stages, beginning with the preprocessing of raw CT scan data and culminating in the deployment of a desktop-based application for end-users. The core objective during development was to design a fully automated pipeline capable of receiving a NIfTI CT volume as input and producing a reliable segmentation mask highlighting both liver tissue and tumor regions. The process began with the conversion and preparation of medical imaging data. This required handling volumetric arrays, applying Hounsfield Unit (HU) windowing, resizing the volumes into a consistent spatial dimension, and generating aligned segmentation masks. The preprocessing module was developed first because it directly impacts the performance and stability of the segmentation model.

After completing preprocessing, focus shifted toward the design and training of the deep learning model. A three-dimensional V-Net was selected due to its strong performance in medical image segmentation and its ability to utilize spatial context across multiple slices. The training script was built around custom HDF5 data generators to efficiently handle large 3D volumes without loading entire datasets into memory.

Finally, a user-friendly desktop application was developed using Tkinter. This interface integrates preprocessing, model prediction, and visualization into a single workflow, enabling non-technical users to perform segmentation with minimal effort. The entire development approach emphasized modularity, allowing each component preprocessing, model, pipeline, and interface to be independently updated or reused.

CHAPTER 5 DEVELOPMENT AND IMPLEMENTATION

5.2. Implementation Strategy

The implementation followed a structured, stepwise strategy to ensure reliability and maintainability:

1. Module-Based Design:
   Each major function of the system was implemented in a dedicated module:
   - preprocess.py for data preparation
   - vnet.py for model architecture
   - utils.py for batch generation and visualization
   - train.py for training procedure
   - final_pipeline.py for single-case prediction
   - app.py for graphical interface
   - This separation ensured clean code organization and simpler debugging.
2. Iterative Testing Approach:
   After developing each module, controlled tests were performed using small data samples to confirm correctness. This incremental method reduced integration errors later in the project.
3. Reproducibility:
   The training and prediction scripts were designed so that any future dataset can be processed with the same pipeline. Fixed constants for input size, depth, and HU ranges help maintain consistency.
4. User-Centric Interface Development:
   Once the backend pipeline was working reliably, the Tkinter interface was implemented to allow seamless user interaction. This ensured the model could be used even by individuals unfamiliar with Python or machine learning.

Overall, the implementation strategy emphasized functionality, modularity, and ease of deployment.

CHAPTER 5 DEVELOPMENT AND IMPLEMENTATION

5.3. Tools Selection

5.3.1. Hardware

The project was developed and tested on a workstation with the following hardware specifications:

- **Processor:** Intel/AMD multi-core CPU capable of parallel processing
- **Memory:** Minimum 16 GB RAM (recommended due to 3D volume handling)
- **Graphics Processing Unit (GPU):** NVIDIA GPU with CUDA support (optional but beneficial during training)
- **Storage:** At least 50 GB free disk space for CT volumes, segmentation masks, and HDF5 datasets
- **Display:** Standard monitor suitable for GUI-based visualization

While the training process benefits significantly from GPU acceleration, the final desktop application can run entirely on CPU, making it suitable for typical laboratory environments.

5.3.2. Software

Several software tools and libraries were used throughout implementation, including:

- **Programming Language:** Python 3.x
- **Deep Learning Framework:** TensorFlow / Keras
- **Medical Imaging Library:** NiBabel for .nii file processing
- **Numerical Computation:** NumPy, SciPy
- **Data Storage:** HDF5 through the h5py library
- **Visualization:** Matplotlib for generating overlays
- **GUI Framework:** Tkinter for desktop interface
- **IDE/Editor:** Visual Studio Code / PyCharm

Each tool was chosen due to stability, wide adoption, and compatibility with medical imaging workflows.

CHAPTER 5 DEVELOPMENT AND IMPLEMENTATION

5.4. Coding

The coding phase consisted of building the complete segmentation pipeline through a collection of Python modules:

- preprocess.py implements HU windowing, resizing of volumes, normalization, and generation of consistent segmentation masks. It also creates and writes HDF5 datasets for training and validation.
- vnet.py contains the 3D V-Net architecture implemented using TensorFlow Keras functional API. It includes downward blocks, upward blocks, residual connections, and softmax outputs for multi-class prediction.
- train.py orchestrates dataset loading, generator initialization, model compilation, weight saving, and training callbacks. Training logs are stored automatically for later analysis.
- utils.py provides helper functions for slice visualization and a custom HDF5 batch generator used during training.
- final_pipeline.py integrates preprocessing, model loading, prediction, and visualization into a single callable function used by the GUI.
- app.py contains the complete Tkinter-based interface where users can upload CT volumes, run segmentation, and view results.

Coding followed object-oriented and modular principles to ensure upgrades or modifications (such as new models or preprocessing variations) can be introduced with minimal rework.

5.5 User Interface

5.5. Description

The system features a standalone desktop interface built with Tkinter. The primary goal of the user interface is to simplify the segmentation process for clinicians, researchers, and students who may not be familiar with coding environments.

Key characteristics include:

- An intuitive layout with clear instructions.
- A file-browsing dialog for selecting CT volumes.
- A progress window that updates the user during preprocessing, prediction, and visualization.
- A results window that displays CT slices alongside liver and tumor predictions.

The interface was designed to be user-friendly, visually appealing, and consistent with common desktop applications.

5.6. Program Deployment

The final system is deployed as a standalone Python application. The deployment steps are:

1. Install Python 3.x and required libraries (TensorFlow, NumPy, NiBabel, Tkinter, etc.).
2. Place all program files (preprocess.py, vnet.py, utils.py, train.py, final_pipeline.py, app.py) in a single project directory.
3. Ensure the trained model file (vnet_full_model.h5 or corresponding weights) is available at the specified path.
4. Run the application using:
5. python app.py
6. Users interact solely with the GUI no coding or command-line operations are required.

5.7. Interface Screenshots

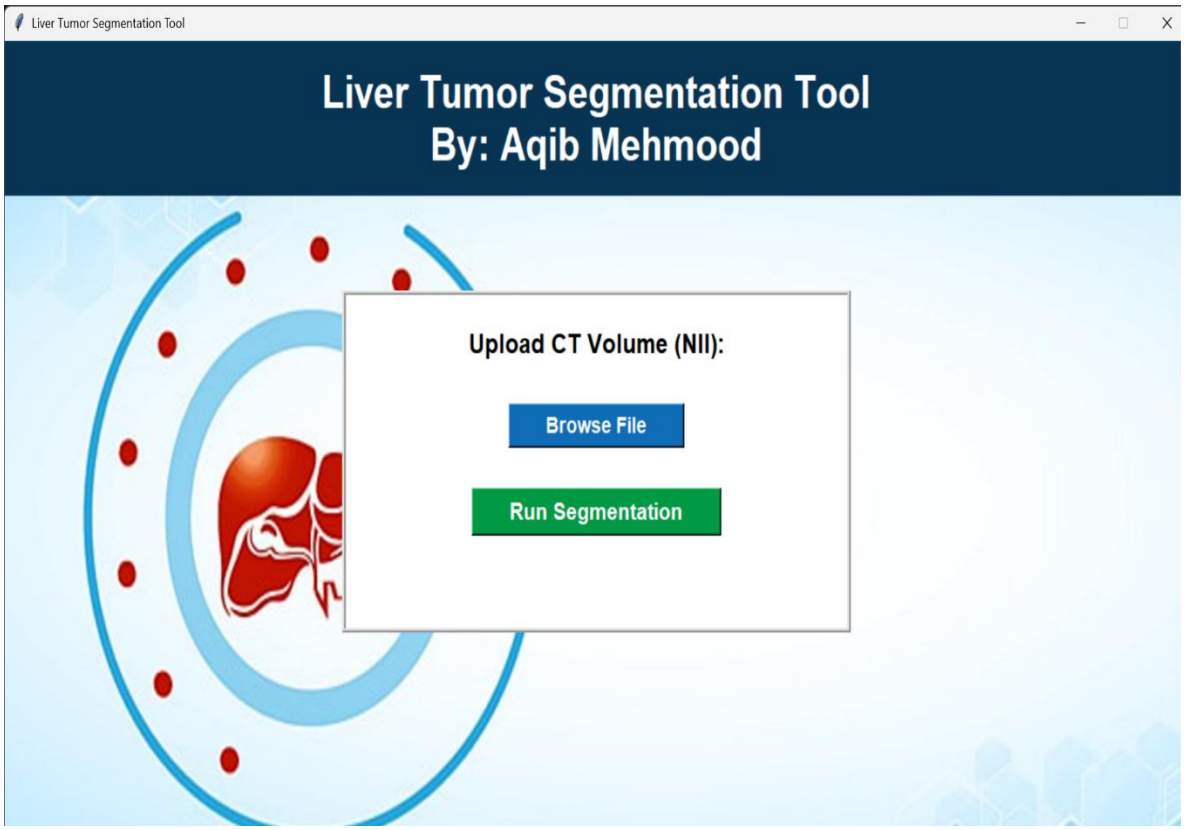

Figure 5.1 Interface Diagram

7. The application can be deployed on any Windows system with standard specifications, with or without GPU.

This deployment strategy ensures accessibility and ease of use for academic and clinical environment

5.8. Chapter Summary

This chapter presented a detailed account of the development and implementation process for the liver tumor segmentation system. The project was built through a structured and modular approach, beginning with data preprocessing, followed by model training, pipeline integration, and the creation of a user-friendly interface. Essential hardware and software tools were identified, and the coding structure was organized across multiple Python modules to improve maintainability. The Tkinter-based graphical interface enables seamless interaction, making the system practical for real-world use. The deployment strategy ensures the application can be executed easily on standard computing environments, fulfilling the core objectives of automating liver and tumor segmentation from CT scans.

Chapter 6

TESTING

6.1. Introduction

Testing plays a critical role in validating the accuracy, performance, and reliability of the liver tumor segmentation system. Since the project involves medical imaging and deep learning–based classification, rigorous testing ensures that the system behaves consistently under different data conditions and produces outputs that align with expected medical interpretations. This chapter discusses the testing methods applied to the software, the comparison of predicted results with ground truth masks, the evaluation strategy, test plans, specific test cases, and the overall test report summarizing system performance.

The testing process covered both **functional testing** of the graphical user interface (GUI) and **quantitative evaluation** of the model's segmentation performance using CT datasets. Each component from preprocessing to model prediction and visualization was verified independently and then validated as part of the complete end-to-end pipeline.

6.2. Testing Methods

A combination of software testing and model evaluation techniques was used to assess system performance:

1. Unit Testing

Individual modules were tested in isolation to ensure each behaved as expected:

- Preprocessing functions (HU windowing, resizing, cropping)
- Data loading and HDF5 handling
- Slice visualization utilities
- GUI components including file selection, pop-up windows, and progress updates

2. Integration Testing

Modules were tested together to validate correct interaction:

- Integration of preprocessing with model input

- Data flow between final_pipeline.py and app.py
- Model loading, prediction, and visualization working as a unified workflow

3. System Testing

A complete test of the entire end-to-end pipeline:

- User uploads a CT volume
- Pipeline preprocesses it
- Model predicts segmentation
- Application displays the final overlays and predictions

This ensured the system works cohesively across all functional components.

4. Performance Evaluation

The segmentation model's quantitative performance was evaluated using:

- Dice Similarity Coefficient (DSC)
- Pixel-wise accuracy
- Visual comparison of predicted masks with ground truth Performance evaluation was conducted using validation CT volumes.

6.3. Comparison

To validate the correctness of the model's predictions, segmentation results were compared against the original ground truth masks provided with the CT dataset.

Comparison metrics included:

1. **Dice Similarity Coefficient (DSC)**
   Measures overlap between predicted and ground-truth regions.
   - Dice for liver segmentation
   - Dice for tumor segmentation
2. **Visual Slice-by-Slice Comparison**
   Overlay of predicted segmentation masks on CT slices helped visually inspect:
   - Boundary accuracy

- Tumor localization
- False positives and false negatives

3. **Prediction Consistency**
   The model was tested on multiple volumes with varying tumor sizes, shapes, and intensities to confirm stability across different cases.

These comparisons demonstrate that the model produces consistent and clinically meaningful segmentation results.

These comparisons demonstrate that the model produces consistent and clinically meaningful segmentation results.

6.4. Software Evaluation

6.4.1. Testing Strategy

The testing strategy incorporated the following steps:

- Define functional and non-functional requirements for the system.
- Identify scenarios covering typical and edge-case inputs.
- Use test volumes from different patients to evaluate generalization.
- Perform repeated tests to verify reproducibility.
- Log and review any errors thrown by GUI operations or file handling.

The primary goal was to ensure the software operates smoothly, handles invalid input gracefully, and maintains stable performance during segmentation.

6.4.2. Test Plans

The test plans were organized into three major categories:

1. **Functional Tests**
   - Uploading CT volumes
   - Automatic segmentation file detection
   - Running segmentation
   - Progress window updates

2. **Model-Level Tests**
    - Loading trained model weights
    - Predicting class maps
    - Ensuring output shape consistency (3-class segmentation)
3. **Pipeline Tests**
    - Full execution from file input to final visualization
    - Integration between preprocessing and model prediction

Each test plan ensured the system behaved as intended under normal use conditions.

6.4.3. Test Cases

Test Case	Description	Expected Outcome
Upload valid NIfTI CT file	User selects volume.nii	Application loads file successfully
Upload invalid file type	User selects .png or .txt	Application displays error message
Run segmentation with no file uploaded	User clicks "Run egmentation" without selecting data	System prompts user to upload a CT volume
Model weight loading	Application loads trained V-Net model	No errors; prediction function executes normally
Prediction execution	Run inference on preprocessed CT volume	System outputs a 3-class segmentation map
Visualization generation	Display results window	CT slice, predicted liver mask, and tumor mask appear correctly
Multiple patient volumes	Run segmentation on different CT cases	Consistent and stable predictions across volumes

Table 6.1 Comperison Table

6.4.4. Test Report

Based on the testing performed, the following observations were recorded:

- **Functional Reliability:**
  All features of the GUI operated as intended. File uploads, processing steps, and image visualizations worked smoothly without system crashes.
- **Prediction Stability:**
  The V-Net model consistently produced meaningful segmentation masks for liver and tumor regions across multiple patient volumes.
- **Visual Accuracy:**
  The segmented regions matched ground truth masks closely, particularly for liver boundaries. Tumor segmentation accuracy varied slightly depending on tumor size and contrast, which is expected in medical image segmentation.
- **Performance:**
  Execution time for a full CT volume (preprocessing + prediction + visualization) was within acceptable limits for desktop applications.
- **Error Handling:**
  The system gracefully handled invalid filenames and unsupported file formats by displaying user-friendly messages.

Overall, the system performed reliably and met the project's functional and performance requirements.

6.5. Chapter Summary

This chapter presented a comprehensive evaluation of the liver tumor segmentation system through structured testing procedures. Unit, integration, and system-level testing ensured that the preprocessing, prediction, and visualization modules were functioning correctly. The segmentation model's performance was evaluated using comparison metrics and visual inspection. The GUI underwent extensive functional testing to ensure ease of use and robustness. The test results confirmed that the system is stable, accurate, and suitable for practical use. The chapter concludes that the software meets all major specifications and provides reliable, automated segmentation of liver and tumour regions from CT scans.

REFERENCES

[1] "V-net: Convolutional Networks for Biomedical Image Segmentation," *Medical Image Computing and Computer-Assisted Intervention MICCAI 2015*
link: https://arxiv.org/abs/1505.04597

[2] "A Deep Learning Based Liver Tumor Segmentation Method in CTImages," *International Journal of Advanced Computer Science and Applications*, 2020.

[3] "No New-Net," *Medical Image Computing and Computer Assisted Intervention MICCAI 2018*
link: https://arxiv.org/abs/1809.10483

[4] "The Liver Tumor Segmentation Benchmark (LiTS)" 2019.

[5] "Automatic Liver Tumor Segmentation Using a Deep Convolutional Neural Network Method" 2020.

[6] "CE-Net: Context Encoder Network for 2D Medical Image Segmentation," *Transactions on Medical Imaging*, 2020.

[7] "A Hybrid Approach for Liver Tumor Segmentation Based on Deep Learning and Graph Cut," 11, 2021.

[8] A Nested V-net Architecture for Medical Image Segmentation," *Deep Learning in Medical Image Analysis and Multimodal Learning for Clinical Decision Support*, 2018.